

Reviewer Pack — Median Half-Subgraph Separator Lower-Bounds Tseitin DPLL Siz...

Entry 118fd4951a23 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 10:32:15 UTC

Median Half-Subgraph Separator Lower-Bounds Tseitin DPLL Size

Entry ID: 118fd4951a23 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 10:32:15 UTC

1. Conjecture

Field A (mathematical branch): Separation profile of geometric group theory (Benjamini-Schramm-Timár 2012; Hume-Mackay-Tessera 2020; Hume 2017): the coarse-geometric invariant $\text{sep}_G(k) = \sup$ over k -vertex subgraphs H of min balanced vertex separator of H , used to distinguish quasi-isometry classes of finitely generated groups. This corner of GGT has essentially no presence in proof complexity — an arXiv search ‘separation profile’ AND (‘resolution’ OR ‘Tseitin’ OR ‘proof complexity’) returns no hits; the closest neighbours are asymptotic-dimension papers that never reach proof systems. **Field B** (complexity object): Tree-like Resolution / DPLL refutation tree size for Tseitin XOR formulas $T(G, \omega)$ on 3-regular graphs, in the Urquhart-Ben-Sasson-Wigderson-Krajíček bounded-arithmetic regime where $\neg T(G, \omega)$ is a Σ^b_0 statement and DPLL size lower-bounds V^0 -witnessing complexity.

Statement:

For a connected 3-regular graph G on m vertices and odd charge ω : $\nu_{30}(G) := \text{median over } 30 \text{ uniformly random vertex subsets } U \subset V(G) \text{ of size } k=\lfloor m/2 \rfloor \text{ of the value } \beta(G[U])$, where $\beta(H)$ is the size of a minimum vertex separator $(A, V(H) \sqcap (A \cup S), S)$ of $G[U]$ with $|A|, |V(H) \sqcap (A \cup S)| \geq \lfloor |V(H)|/3 \rfloor$, and $\beta(H) := 0$ if $G[U]$ is disconnected. Conjecture: (i) every odd-charge Tseitin formula $T(G, \omega)$ requires DPLL tree size $s(T(G, \omega)) \geq 2^{\{0.1 \cdot \nu_{30}(G)\}}$; (ii) for every 3-regular G admitting a vertex partition (V_1, V_2) with $|V_1|, |V_2| \geq m/3$ and $|E(V_1, V_2)| \leq 2$ (e.g., barbells, prism-pairs joined by a bridge or two edges), $\nu_{30}(G) \leq 2$. A single 3-regular instance with $\log_2(s) < 0.1 \cdot \nu_{30}(G)$, or a small-cut graph in class (ii) with $\nu_{30}(G) > 2$, refutes the conjecture.

Rationale (proposer’s reasoning):

The Ben-Sasson-Wigderson width method bounds Tseitin Resolution length via expansion of G as a whole; but bounded arithmetic V^0 -witnessing of unsatisfiable Σ^b_0 -instances should react to *every scale* of expansion, not just the global one. The separation profile evaluated at half-size captures exactly this: it is large iff *typical* half-subgraphs of G are themselves robustly connected, which is the structural data DPLL must repeatedly re-encounter on any branch that fixes half the variables. The link to Cook-Reckhow-Krajíček: ν_{30} is a low-complexity (NC-computable up to sampling) Σ^b_0 -definable property of G , so a separation ‘ ν_{30} large $\Rightarrow$ refutation long’ is exactly the bounded-arithmetic style of structural lower bound that escapes both NATURAL PROOFS (ν_{30} depends on G , not on a truth-table) and ALGEBRIZATION (no ring extension respects vertex-separator counts).

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cfdc54676ce9a356

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30-seeded random 3-regular graphs at $m \in \{8, 10, 12, 14, 16, 18, 20\}$ plus barbell anchors at $m \in \{8, 12, 16, 20\}$: Spearman ρ between $\log_2(s)$ and $v_{30}(G) \geq 0.6$; every instance satisfies $\log_2(s) \geq 0.1 \cdot v_{30}(G)$ (clause i); every anchor satisfies $v_{30}(G) \leq 2$ (clause ii).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formula resolution lower bound vertex separator expansion - DPLL tree size Tseitin XOR formula balanced separator 3-regular - separation profile graph proof complexity resolution width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import deque

def generate_3_regular_graph(n, seed):
    random.seed(seed)
    if n % 2 != 0:
        raise ValueError("n must be even for 3-regular graph")
    stubs = list(range(n)) * 3
    edges = set()
    while stubs:
        u = stubs.pop()
        v = random.choice([x for x in stubs if x != u])
        stubs.remove(v)
```

```

        edges.add(frozenset({u, v}))
    return edges

def is_connected(edges, n):
    if not edges:
        return False
    graph = {i: set() for i in range(n)}
    for u, v in edges:
        graph[u].add(v)
        graph[v].add(u)
    visited = set()
    queue = deque([0])
    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            queue.extend(graph[node] - visited)
    return len(visited) == n

def compute_min_vertex_separator(edges, n):
    if not is_connected(edges, n):
        return 0
    min_separator = float('inf')
    for k in range(1, n):
        for subset in itertools.combinations(range(n), k):
            A = set(subset)
            S = set()
            for u in A:
                for v in range(n):
                    if v not in A and v not in S:
                        if v in [x for x in range(n) if any((u, x) in edges or (x, u) in edges)]:
                            S.add(v)
            if len(A) >= n // 3 and len(set(range(n)) - A - S) >= n // 3:
                min_separator = min(min_separator, len(S))
    return min_separator if min_separator != float('inf') else 0

def compute_nu_30(edges, n):
    k = n // 2
    nu_values = []
    for _ in range(30):
        U = random.sample(range(n), k)
        subgraph_edges = [edge for edge in edges if edge.issubset(U)]
        nu_values.append(compute_min_vertex_separator(subgraph_edges, k))
    return sum(nu_values) / len(nu_values)

def generate_tseitin_cnf(edges, n, seed):
    random.seed(seed)
    omega = [random.choice([-1, 1]) for _ in range(n)]
    clauses = []
    for u, v in edges:
        x = f"x_{u}_{v}"
        clauses.append([x, f"v_{u}", f"v_{v}"])
        clauses.append([f"-{x}", f"v_{u}", f"-v_{v}"])
        clauses.append([f"-{x}", f"-v_{u}", f"v_{v}"])
        clauses.append([x, f"-v_{u}", f"-v_{v}"])
    for i in range(n):

```

```

    if omega[i] == 1:
        clauses.append([f"v_{i}"])
    else:
        clauses.append([f"-v_{i}"])
return clauses

def dpll_satisfiable(clauses, max_nodes=10**6):
    assignments = {}
    nodes = 0

    def unit_propagate():
        nonlocal nodes
        changed = True
        while changed:
            changed = False
            for clause in clauses:
                unassigned = [lit for lit in clause if lit not in assignments]
                if len(unassigned) == 1:
                    lit = unassigned[0]
                    if lit.startswith('-'):
                        assignments[lit[1:]] = False
                    else:
                        assignments[lit] = True
                    changed = True
                    nodes += 1
                    if nodes > max_nodes:
                        return False
            return True

    def dpll():
        nonlocal nodes
        if not unit_propagate():
            return False, nodes
        for clause in clauses:
            if all(lit in assignments and (lit.startswith('-') != assignments.get(lit[1:], None))
                    for lit in clause):
                return False, nodes
        if all(lit in assignments for clause in clauses for lit in clause):
            return True, nodes
        for clause in clauses:
            for lit in clause:
                if lit not in assignments:
                    if lit.startswith('-'):
                        var = lit[1:]
                    else:
                        var = lit
                    if var not in assignments:
                        assignments[var] = True
                        satisfied, nodes = dpll()
                        if satisfied:
                            return True, nodes
                        assignments[var] = False
                        satisfied, nodes = dpll()
                        if satisfied:
                            return True, nodes
                    del assignments[var]
        return False, nodes

```

```

        return False, nodes

satisfied, nodes = dpll()
return math.log2(nodes) if satisfied else float('inf')

def run_trial(seed):
    n_values = [8, 10, 12, 14, 16, 18, 20]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        try:
            edges = generate_3_regular_graph(n, seed)
            if not is_connected(edges, n):
                continue
            nu_30 = compute_nu_30(edges, n)
            cnf = generate_tseitin_cnf(edges, n, seed)
            log2_s = dpll_satisfiable(cnf)
            metric_values.append(log2_s)
            instances_tested += 1

            if log2_s < 0.1 * nu_30:
                conjecture_holds = False
                counterexample = f"Instance with n={n}, log2(s*)={log2_s}, nu_30={nu_30}"
                break
        except IndexError:
            continue

    if conjecture_holds:
        for n in [8, 12, 16, 20]:
            try:
                edges = generate_3_regular_graph(n, seed)
                if not is_connected(edges, n):
                    continue
                nu_30 = compute_nu_30(edges, n)
                if nu_30 > 2:
                    conjecture_holds = False
                    counterexample = f"Anchor instance with n={n}, nu_30={nu_30}"
                    break
            except IndexError:
                continue

    return {
        "metric_name": "log2(s*)",
        "metric_value": sum(metric_values) / len(metric_values) if metric_values else 0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    result["seed"] = seed
    print(f"TRIAL: {result}")
    results.append(result)

metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
if metric_values:
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
else:
    mean, std = 0, 0

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41e6d60a.py", line 194, in <module>
 result = run_trial(seed)
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_41e6d60a.py", line 155, in run\_trial  
 nu\_30 = compute\_nu\_30(edges, n)  
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41e6d60a.py", line 68, in compute_nu_30
 nu_values.append(compute_min_vertex_separator(subgraph_edges, k))
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_41e6d60a.py", line 46, in compute\_min\_v  
 if not is\_connected(edges, n):  
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41e6d60a.py", line 35, in is_connected
 graph[v].add(u)
 ~~~~~

KeyError: 7

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a `KeyError` in `is_connected` before producing any `RESULT` line, so neither support nor falsification conditions can be evaluated. The critic also challenged due to lack of data. | next: Fix the graph construction in `is_connected` (initialize adjacency entries for all vertices in the induced subgraph, e.g., using `defaultdict(set)` or pre-populating keys) and rerun the pre-registered protocol.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider   | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max | opus             | 0         | 0   | 154325       |
| 2 | preregistration | claude_max | opus             | 0         | 0   | 5430         |
| 3 | novelty         | claude_max | opus             | 0         | 0   | 3511         |
| 4 | novelty         | claude_max | opus             | 0         | 0   | 7579         |
| 5 | test_gen        | mistral    | codestral-latest | 0         | 0   | 317027       |
| 6 | test_gen        | mistral    | codestral-latest | 0         | 0   | 314103       |
| 7 | test_gen        | mistral    | codestral-latest | 0         | 0   | 317602       |
| 8 | test_gen        | mistral    | codestral-latest | 0         | 0   | 313334       |
| 9 | judge           | claude_max | opus             | 0         | 0   | 4228         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1437138 ms total latency. Provider mix: {'claude\_max': 5, 'mistral': 4}

*(full prompt+response transcripts available in `research/audit/118fd4951a23.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/118fd4951a23.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/118fd4951a23.tar.gz` (if generated)